Approximation Algorithms

CLRS 4th Edition, Chapter 35

Algorithm Engineering

11 Aug 2026

Lecture map

1. Why approximate?

NP-complete optimization problems can be computationally out of reach when exact optimality is required.

2. Vertex Cover

A simple greedy construction gives a 2-approximation.

3. Metric TSP

An MST plus shortcutting gives a 2-approximation when the triangle inequality holds.

4. Set Cover

Greedy selection gives a harmonic-number approximation.

Key Idea

The recurring proof pattern is:

upper-bound the algorithm + lower-bound OPT.

1. Why Approximation?

Trade optimality for a guarantee

The optimization problem we actually face

Suppose a problem asks for a feasible solution of minimum cost.

Exact optimization

Find a feasible solution S^* such that

$$\text{cost}(S^*) = \text{OPT} = \min\{\text{cost}(S) : S \text{ feasible}\}.$$

For many important problems, exact optimization is computationally difficult.

- We may still need an answer for large instances.
- A heuristic can be fast, but may provide no worst-case guarantee.
- An approximation algorithm deliberately accepts a nonoptimal solution *with a provable bound*.

Key Idea

Approximation is not "guess and hope." The quality guarantee is part of the algorithm.

Approximation algorithm: the contract

For an input of size n, let

$$C = \text{cost of the algorithm's solution}, \qquad C^* = \text{cost of an optimal solution}.$$

Approximation ratio

A $\rho(n)$-approximation algorithm produces a solution whose cost satisfies

$$\max\left\{\frac{C}{C^*}, \frac{C^*}{C}\right\} \leq \rho(n).$$

For a minimization problem:

$$C \geq C^* \implies \boxed{C/C^* \leq \rho(n)}.$$

For a maximization problem:

$$C \leq C^* \implies \boxed{C^*/C \leq \rho(n)}.$$

- $\rho(n) = 1$: exact optimization.
- Smaller ρ: stronger guarantee.
- A constant ρ: quality degrades by at most a fixed factor.

Ratio versus heuristic quality

Heuristic

A method may work extremely well on typical instances but have no useful worst-case bound.

Approximation algorithm

We can say something like:

$$\mathrm{ALG} \leq 2\mathrm{OPT}$$

for *every* input instance.

Key Idea

The guarantee is a theorem about all inputs, not an empirical observation.

The central question

How do we prove a relationship between ALG and OPT without knowing OPT explicitly?

A proof strategy to remember

For minimization problems, the target is usually

$$\mathrm{ALG} \leq \rho\,\mathrm{OPT}.$$

The algorithm itself gives an upper bound on ALG.
The analysis searches for a structural lower bound on OPT:

$$\underbrace{\text{something we can compute or reason about}}_{\text{lower bound}} \leq \mathrm{OPT}.$$

Then combine:

$$\mathrm{ALG} \leq \rho \cdot (\text{lower bound}) \leq \rho\,\mathrm{OPT}.$$

Key Idea

Good approximation proofs often avoid computing OPT. Instead, they prove that OPT must be large enough to "pay for" what the algorithm does.

2. Vertex Cover

Greedy edges + a matching lower bound

Vertex Cover problem

Let $G = (V, E)$ be an undirected graph.

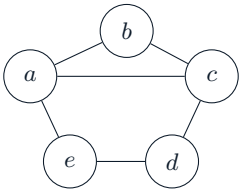
Definition

A set $C \subseteq V$ is a vertex cover if every edge in E has at least one endpoint in C.

Optimization objective

Find a vertex cover of minimum cardinality:

$$\mathrm{OPT} = \min\{|C| : C \subseteq V,\ \forall (u,v) \in E,\ \{u,v\} \cap C \neq \varnothing\}.$$



A simple greedy idea

If an uncovered edge (u, v) remains, then *any* vertex cover must contain at least one of u or v.
So select both endpoints:

$$C \leftarrow C \cup \{u, v\}.$$

Then remove all edges incident to u or v and continue.

APPROX-VERTEX-COVER

1: $C \leftarrow \varnothing$
2: $E' \leftarrow E$
3: **while** $E' \neq \varnothing$ **do**
4: choose any edge $(u, v) \in E'$
5: $C \leftarrow C \cup \{u, v\}$
6: remove from E' every edge incident to u or v
7: **end while**
8: **return** C

Complexity

With suitable graph representations, the procedure is polynomial time.

Why is the output a vertex cover?

At termination, no edge remains in E'.
Consider an original edge (x, y).

1. If it was selected, both endpoints were inserted into C.
2. Otherwise, it was removed because one of its endpoints was selected into C.

Therefore every original edge has at least one endpoint in C:

$$\boxed{C \text{ is a vertex cover.}}$$

Key Idea

Correctness and approximation quality are separate claims:

$$\text{first prove feasibility; then prove } |C| \leq 2\mathrm{OPT}.$$

The hidden structure: selected edges form a matching

Let

$$M = \{e_1, e_2, \ldots, e_k\}$$

be the edges selected by the algorithm.
Once an edge (u, v) is selected, all edges incident to u or v are removed. Therefore no later selected edge shares an endpoint with it.

Matching

A set of edges is a matching if no two edges share a vertex.

Hence

$$M \text{ is a matching} \quad \Longrightarrow \quad |M| = k.$$

The algorithm adds exactly two vertices per selected edge:

$$\boxed{|C| = 2k.}$$

The lower bound on OPT

Every edge in a matching has two endpoints, and the edges are vertex-disjoint.
A vertex cover must touch *every* edge in M.
Because the edges in M are disjoint, one vertex cannot cover two different matching edges.
Therefore any vertex cover needs at least one vertex per edge of M:

$$\boxed{\mathrm{OPT} \geq |M| = k.}$$

Combine with the algorithm's cost:

$$|C| = 2k \leq 2\mathrm{OPT}.$$

Key Idea

The matching is a certificate that OPT cannot be too small.

Vertex Cover: the complete 2-approximation proof

1. The algorithm returns a valid vertex cover C.
2. Let M be the set of edges selected by the algorithm.
3. M is a matching because selected edges never share endpoints.
4. Any vertex cover must contain at least one endpoint of every edge in M.
5. Since M is a matching:
$$\mathrm{OPT} \geq |M|.$$
6. The algorithm selects both endpoints of each edge in M:
$$|C| = 2|M|.$$

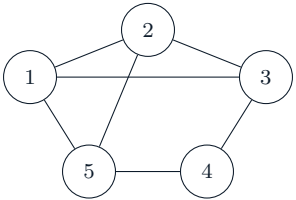
Thus

$$\boxed{|C| \leq 2\mathrm{OPT}.}$$

Conclusion

Approx-Vertex-Cover is a polynomial-time 2-approximation algorithm.

Worked example: what the algorithm pays for



Suppose the algorithm first selects $(1, 2)$.

- Add $1, 2$ to C.
- Remove every edge incident to 1 or 2.
- The remaining uncovered edge(s) can be handled next.

The selected edges are automatically disjoint.

Key Idea

Do not try to guess which endpoint is "better." The proof deliberately pays for both endpoints and charges those two vertices to one forced edge.

When is the 2 bound tight?

Consider a graph consisting of disjoint edges:

$$(u_1, v_1), (u_2, v_2), \ldots, (u_k, v_k).$$

OPT

One endpoint from each edge:

$$\mathrm{OPT} = k.$$

Greedy algorithm

Selecting an edge adds both endpoints:

$$\mathrm{ALG} = 2k.$$

Thus

$$\frac{\mathrm{ALG}}{\mathrm{OPT}} = 2.$$

Key Idea

The analysis is not merely a loose algebraic trick: there are instances where this particular algorithm really reaches its factor 2.

3. TSP with Triangle Inequality

Build cheaply, traverse completely, then shortcut

The Traveling-Salesman Problem

Given a complete undirected graph with edge costs $c(u, v)$, find a minimum-cost tour that

1. starts at a city,
2. visits every city exactly once,
3. returns to the start.

Let OPT denote the minimum tour cost.

Why is TSP difficult?

The exact optimization problem is computationally hard. Chapter 35 therefore studies a restricted version where the edge costs satisfy the triangle inequality.

$$\boxed{c(u, w) \leq c(u, v) + c(v, w)}$$

for every triple of cities u, v, w.

Why the triangle inequality changes everything

Suppose a tour traversal visits

$$u \to v \to w$$

but we want to skip v.
The triangle inequality gives

$$c(u, w) \leq c(u, v) + c(v, w).$$

So replacing the two-edge detour by the direct edge cannot increase the cost.

Key Idea

Triangle inequality makes shortcutting safe.

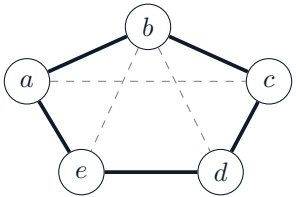
This is the crucial bridge from a tree traversal, which may revisit vertices, to a valid TSP tour, which visits each vertex once.

The MST lower bound

Let T be a minimum spanning tree (MST) of the complete weighted graph.
Take an optimal TSP tour and remove one edge from it.
The remaining edges form a spanning tree, so:

$$\boxed{w(T) \leq \mathrm{OPT}.}$$

This is the lower-bound half of the approximation proof.



MST doubling: the construction

Let T be an MST.

1. Compute an MST T.
2. Duplicate every edge of T.
3. The resulting multigraph has an Euler tour.
4. Traverse the Euler tour.
5. Shortcut repeated vertices.

The doubled tree has total weight

$$2w(T).$$

The Euler tour therefore costs exactly

$$2w(T).$$

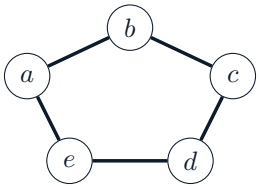
Because of the triangle inequality, shortcutting does not increase cost.

Why doubling creates an Euler tour

Every vertex of a tree has even degree after every edge is duplicated:

$$\deg_{2T}(v) = 2\deg_T(v).$$

The doubled graph is connected and all degrees are even.
Therefore it has an Euler tour: a closed walk that uses every duplicated edge exactly once.



Preorder walk and shortcutting

A depth-first/preorder traversal of the MST may look like

$$a \to b \to c \to b \to d \to e \to d \to a.$$

Repeated vertices are not allowed in a TSP tour.
Shortcut:

$$a \to b \to c \to d \to e \to a.$$

Every shortcut uses

$$c(x, z) \leq c(x, y) + c(y, z).$$

Thus:

$$\boxed{\text{shortcut tour cost} \leq 2w(T).}$$

Key Idea

The algorithm does not need the MST itself to be a tour. It uses the MST as a cheap "skeleton" from which a tour can be constructed.

Metric TSP: the 2-approximation proof

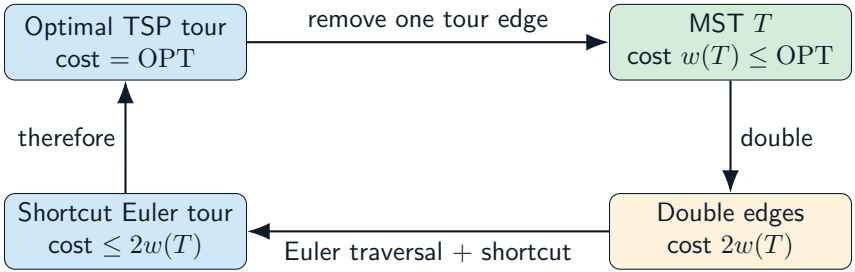
Let C be the tour returned by the algorithm.

$$
\begin{aligned}
w(T) &\leq \mathrm{OPT} && \text{(MST is no more expensive than an optimal tour)},\\
2w(T) &\leq 2\mathrm{OPT}, &&\\
w(C) &\leq 2w(T) && \text{(Euler traversal + shortcutting)},\\
\therefore \quad w(C) &\leq 2\mathrm{OPT}. &&
\end{aligned}
$$

Theorem

For TSP instances satisfying the triangle inequality, MST doubling gives a polynomial-time 2-approximation algorithm.

The entire proof in one picture



The final edge of reasoning is:

$$w(C) \leq 2w(T) \leq 2\mathrm{OPT}.$$

Why general TSP has no constant-factor approximation

Drop the triangle inequality.
Reduce from Hamiltonian-Cycle.
For a graph G on n vertices, construct a complete graph with

$$c(u,v) = \begin{cases} 1, & (u,v) \in E(G), \\ M, & (u,v) \notin E(G), \end{cases}$$

where M is chosen much larger than any proposed constant ratio.

- If G has a Hamiltonian cycle, there is a tour of cost n.
- If G has no Hamiltonian cycle, every tour uses a non-edge and costs at least $M + (n-1)$.

For a fixed approximation factor ρ, choose M so large that the two cases can be distinguished by the approximation output.

Consequence

Unless $\mathrm{P} = \mathrm{NP}$, general TSP has no polynomial-time constant-factor approximation.

The lesson from TSP

Without triangle inequality

Shortcutting may be arbitrarily expensive.

$$c(u, w) \gg c(u, v) + c(v, w)$$

can happen.

With triangle inequality

Shortcutting is safe:

$$c(u, w) \leq c(u, v) + c(v, w).$$

Key Idea

A small structural assumption can turn an inapproximable optimization problem into one with a simple constant-factor approximation.

Beyond today's CLRS treatment

Christofides' algorithm improves the factor for metric TSP to $3/2$. We mention it here only as a pointer, not as part of today's proof.

4. Set Cover

Greedy coverage + charging

Set-Covering problem

Let

$$X = \text{universe of elements}, \qquad \mathcal{F} = \{S_1, S_2, \ldots, S_m\}, \quad S_i \subseteq X.$$

A family $\mathcal{C} \subseteq \mathcal{F}$ is a set cover if

$$\bigcup_{S \in \mathcal{C}} S = X.$$

Optimization objective

Minimize the number of selected sets:

$$\mathrm{OPT} = \min\{|\mathcal{C}| : \mathcal{C} \subseteq \mathcal{F}, \ \bigcup_{S \in \mathcal{C}} S = X\}.$$

Key Idea

The cost is the number of sets selected, *not* the number of elements.

Greedy Set Cover

Maintain the uncovered set U.
At each step choose the set covering the largest number of currently uncovered elements:

$$S \in \arg\max_{F \in \mathcal{F}} |F \cap U|.$$

Then update

$$U \leftarrow U \setminus S.$$

GREEDY-SET-COVER

1: $U \leftarrow X$
2: $\mathcal{C} \leftarrow \varnothing$
3: **while** $U \neq \varnothing$ **do**
4: choose $S \in \mathcal{F}$ maximizing $|S \cap U|$
5: $U \leftarrow U \setminus S$
6: $\mathcal{C} \leftarrow \mathcal{C} \cup \{S\}$
7: **end while**
8: **return** $\mathcal{C}$

A concrete greedy run

Let

$$X = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

and

$$S_1 = \{1, 2, 3, 4\}, \quad S_2 = \{3, 4, 5, 6\},$$
$$S_3 = \{5, 6, 7\}, \quad S_4 = \{1, 7, 8\},$$
$$S_5 = \{2, 4, 6, 8\}.$$

Initially $U = X$.

1. S_1 and S_2 cover 4 elements: choose one (tie-breaking matters).
2. Suppose S_1 is chosen. Then $U = \{5, 6, 7, 8\}$.
3. S_2, S_3, S_5 each cover 2 uncovered elements: choose by tie-break.
4. Continue until $U = \varnothing$.

Key Idea

The greedy rule is local: maximize immediate new coverage. The approximation proof explains why this local rule is globally controlled.

Why a logarithm appears

Suppose r elements remain uncovered and an optimal solution uses k sets.
Those k optimal sets cover all r remaining elements.
Therefore at least one of them covers at least

$$\frac{r}{k}$$

of the remaining elements.
Greedy chooses a set covering at least that many:

$$r_{\text{next}} \leq r - \frac{r}{k} = r\left(1 - \frac{1}{k}\right).$$

Repeatedly,

$$r_t \leq |X|\left(1 - \frac{1}{k}\right)^t \leq |X|e^{-t/k}.$$

So the number of greedy rounds is

$$O(k \ln |X|).$$

Key Idea

This gives the intuition for the logarithmic approximation factor.

A sharper CLRS proof: charge each element

The greedy algorithm selects sets

$$S_1, S_2, \ldots, S_k.$$

When S_i is selected, let

$$a_i = |S_i \setminus (S_1 \cup \cdots \cup S_{i-1})|$$

be the number of elements newly covered.
Assign each newly covered element charge

$$c_x = \frac{1}{a_i}.$$

Then each selected set contributes total charge

$$a_i \cdot \frac{1}{a_i} = 1.$$

Hence

$$\boxed{|\mathcal{C}| = \sum_{x \in X} c_x.}$$

This converts "number of selected sets" into a sum over elements.

The key charging inequality

Fix any set $S \in \mathcal{F}$.
Suppose its elements become covered over several greedy iterations.
When an element of S is covered, greedy's chosen set covers at least as many *currently uncovered* elements as S does.
If r elements of S remain uncovered just before an iteration, the charge to the next elements of S is at most

$$\frac{1}{r}.$$

Across the possible values

$$|S|, |S|-1, \ldots, 1,$$

the total charge assigned to elements of S is at most

$$1 + \frac{1}{2} + \cdots + \frac{1}{|S|} = H(|S|).$$

Thus:

$$\boxed{\sum_{x \in S} c_x \leq H(|S|).}$$

From one set to an optimal cover

Let $\mathcal{C}^*$ be an optimal cover.
Because $\mathcal{C}^*$ covers every element,

$$\sum_{x \in X} c_x \leq \sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x.$$

Using the key inequality:

$$\sum_{x \in X} c_x \leq \sum_{S \in \mathcal{C}^*} H(|S|) \leq |\mathcal{C}^*| \, H(d),$$

where

$$d = \max_{S \in \mathcal{F}} |S|.$$

But

$$|\mathcal{C}| = \sum_{x \in X} c_x.$$

Therefore:

$$\boxed{|\mathcal{C}| \leq H(d) \, |\mathcal{C}^*|.}$$

Harmonic numbers

The dth harmonic number is

$$H(d) = \sum_{i=1}^{d} \frac{1}{i}.$$

For example:

$$H(1) = 1, \qquad H(2) = \frac{3}{2}, \qquad H(3) = \frac{11}{6}, \qquad H(4) = \frac{25}{12}.$$

A standard bound is

$$H(d) \leq \ln d + 1.$$

Since $d \leq |X|$,

$$H(d) \leq \ln |X| + 1.$$

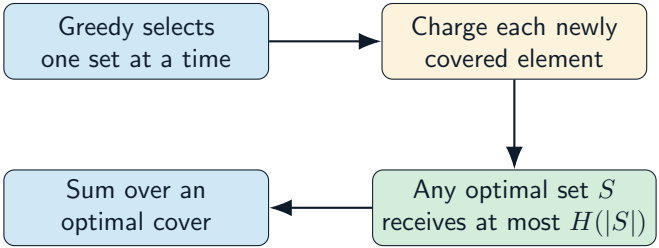
CLRS guarantee

GREEDY-SET-COVER is a polynomial-time

$$H(d)\text{-approximation},$$

and therefore also an $(\ln |X| + 1)$-approximation.

Set Cover: the proof pattern



The final inequality is

$$\mathrm{ALG} = \sum_{x \in X} c_x \leq \sum_{S \in \mathcal{C}^*} H(|S|) \leq H(d)\mathrm{OPT}.$$

Key Idea

Charging is a systematic way to make a greedy proof pay for itself.

5. The Common Pattern

Three problems, three structural lower bounds

Three approximation algorithms side by side

Problem	Algorithmic idea	Lower bound on OPT	Guarantee
Vertex Cover	Pick an uncovered edge; add both endpoints	Selected edges form a matching; each needs a cover vertex	2
Metric TSP	MST → double → Euler walk → shortcut	$w(\mathrm{MST}) \leq \mathrm{OPT}$	2
Set Cover	Pick set covering most uncovered elements	Charging against any optimal cover	$H(d) \leq \ln \lvert X \rvert + 1$

Key Idea

The algorithms look different. The analysis repeatedly asks the same question:

What unavoidable structure must OPT pay for?

A reusable approximation-proof checklist

When you meet a new approximation algorithm, ask:

1. **Feasibility:** Does the algorithm always return a valid solution?
2. **Polynomial time:** Can every step be implemented efficiently?
3. **What does the algorithm pay?** Express ALG structurally.
4. **What must OPT pay?** Find a lower bound.
5. **Can I compare the two?**
6. **Where is the problem's structure used?**

Today's three answers

- Vertex Cover $\rightarrow$ matching.
- Metric TSP $\rightarrow$ MST + triangle inequality.
- Set Cover $\rightarrow$ harmonic charging.

Common mistakes

Mistake 1: "It seems close, so it is an approximation."

Empirical closeness is not a worst-case approximation guarantee.

Mistake 2: Forgetting which direction the inequality goes

For minimization:

$$\mathrm{ALG} \geq \mathrm{OPT}, \qquad \mathrm{ALG} \leq \rho \mathrm{OPT}.$$

Mistake 3: Using shortcutting without triangle inequality

Without the triangle inequality, shortcutting can increase cost dramatically.

Mistake 4: Charging the wrong quantity

In Set Cover, the cost is the number of selected sets; charges must sum to that cost.

Exam-style questions to practice

1. Given a graph, execute Approx-Vertex-Cover and identify the matching produced by the selected edges.
2. Prove $|C| \leq 2\mathrm{OPT}$ using that matching.
3. Explain exactly where the triangle inequality is used in metric TSP.
4. Prove $w(\mathrm{MST}) \leq \mathrm{OPT}$ for metric TSP.
5. For a small set-cover instance, execute greedy set cover and compute its ratio if an optimal cover is known.
6. Reproduce the charging argument leading to $H(d)$.
7. Explain why a constant-factor approximation for general TSP would imply $\mathrm{P} = \mathrm{NP}$.

Key Idea

In an exam, do not only state the ratio. Be able to identify the structural reason that makes the ratio provable.

Take-home summary

Vertex Cover

$$\boxed{\mathrm{ALG} \leq 2\mathrm{OPT}}$$

Matching lower bound.

Metric TSP

$$\boxed{\mathrm{ALG} \leq 2\mathrm{OPT}}$$

MST lower bound + safe shortcutting.

Set Cover

$$\boxed{\mathrm{ALG} \leq H(d)\mathrm{OPT}}$$

Charging argument.

Key Idea

Approximation algorithms replace an impossible demand — "always find OPT" — with a rigorous promise:

$$\boxed{\text{find a good solution, and prove how good it is.}}$$

Reference and scope

Primary reference

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022, Chapter 35: *Approximation Algorithms*.

Covered in this lecture

- Approximation ratios and performance guarantees.
- §35.1: Vertex Cover.
- §35.2: TSP, with emphasis on the triangle-inequality case and the inapproximability of general TSP.
- §35.3: Set Cover and the greedy $H(d)$ guarantee.

Scope note

Christofides' $3/2$ result is mentioned only as an external pointer; its proof is not part of the CLRS Chapter 35 material developed in these slides.

When exactness is too expensive, prove how close you can get.

Vertex Cover — Metric TSP — Set Cover